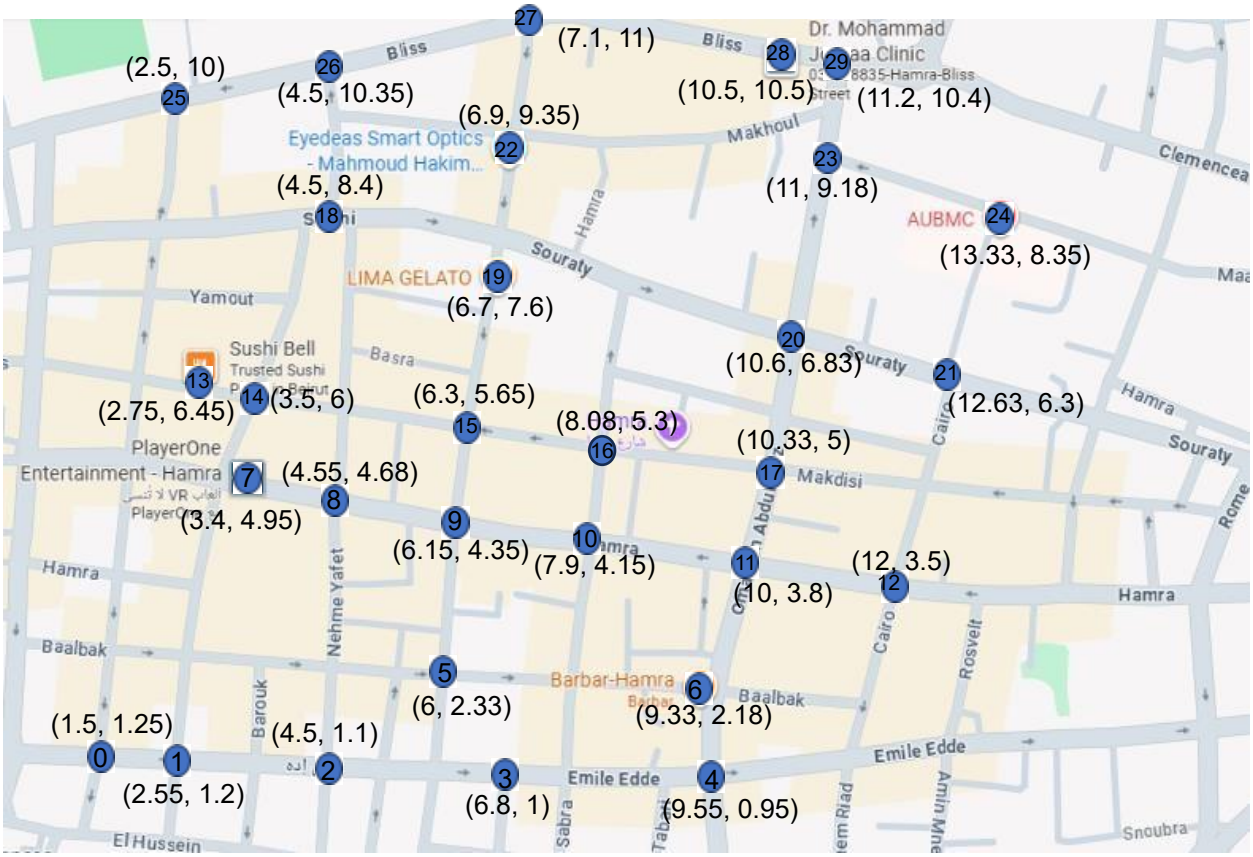

I. Description:

This project aims to find the shortest path from a start to destination, both user-defined, in a small area of Hamra's streets. An undirected graph was created by selecting 30 vertices found on the map and copying the edges that exist between these vertices. The choice of undirected graph was to simplify modeling and reduce edge complexity. The purpose of this project is not to find the exact real-world shortest path of this area, but to illustrate the concept of shortest path algorithms and their application on a real map.

II. Creating Vertices:

Vertices were chosen in a way to include all recognizable structures (sushi bell, barbar, …) and intersections between these structures in such that they are reachable in an undirected graph. The coordinates are in centimeters, considering the bottom left corner of the map to be the origin point (0,0). The built-in word ruler at the top and left side of the page were used to calculate the x and y coordinates. These values are approximate and do not represent exact real-world positions.

III. Algorithm:

Several points were created each with a name, a kind (restaurant, building, intersection, …) and coordinates from the map. All the edges found on the map were also added (that is, an edge was created between any two vertices that have a crossable path between them). Distances between vertices are computed using **Euclidean distance** and scaled to approximate real-world meters.

The program uses **Dijkstra's algorithm** learned in class whereby the weight is the distance between the vertices. For simplicity, a basic array-based implementation of Dijkstra's algorithm is used instead of a priority queue. The algorithm selects the next closest unvisited node by scanning all vertices and choosing the one with the smallest current distance.

After the algorithm is done, the path is constructed using a parent array and interpreted from the start vertex to the destination in order. At each step, an instruction is displayed which tells the user which direction to go (based on coordinate differences), where to go (AUBMC, bliss 1, …), and for how many meters.

The direction is chosen is by checking along which axis we moved more, the x-axis or y-axis? If it is x-axis, then the option is either right or left, otherwise the option is between up or down. Then we check the sign of displacement and display right/up if positive and left/down if negative.

IV. Program:

1) The user is continuously prompted to input a start and destination until the program receives valid vertex names found inside the graph. (inputs are not case sensitive) - The case where the start and destination are the same vertex is handled.

```
Please enter Start Location: aubmc
Please enter Destination: abc
No such vertex. Please reenter destination: AUbMc
You are already at your destination
```

2) Once valid inputs are provided, the shortestPath(start, dest) method is called.
3) The approximate destination distance is first displayed.
4) The program provides instructions on how to reach destination, 1 direction at a time.

- A counter is used to display each direction once if the shortest path consists of 2 or more adjacent vertices that are in the same direction
(instead of "go up 10m to x, go up 15m to y", "go up 25m to y" is displayed)

Once the destination is reached, an appropriate message is displayed and the program terminates successfully.

```
Please enter Start Location: sushi bell
Please enter Destination: barbar

Destination is approximately 101m away:
Directions:
Go right 8 meters to Makdisi 1 Intersection
Go down 10 meters to PlayerOne Entertainment Arcade
Go right 27 meters to Hamra 1 Intersection
Go down 20 meters to Baalbak Intersection
Go right 33 meters to Barbar Restaurant
You have reached your destination!
```